



MobileFaceNet Convolutional Frontend Accelerator

ASIC (FreePDK45) and FPGA Co-Implementation

CE392 VLSI Design Projects — Final Report

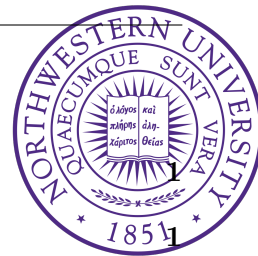
Willie Liao — GEL8580

Peixin Zhang — JZH2008

Instructor: Prof. David Zaretsky

Northwestern University

June 2, 2026



Contents

Executive Summary

1	Introduction	
2	Model and Numerical Design	2
2.1	MobileFaceNet Topology	2
2.2	Precision Selection	3
2.3	Model Evaluation	3
3	Broader Considerations	4
4	Design Constraints and Requirements	4
4.1	Process and IP Availability	5
4.2	Verification Capability	5
5	Hardware Architecture	5
5.1	Architecture Overview	5
5.2	Frontend Design (RTL Architecture)	6
5.2.1	ASIC Frontend (Liao)	6
5.2.2	FPGA Frontend (Zhang)	7
5.3	Backend Design (Physical Implementation)	8
5.3.1	ASIC Backend (Liao): Genus Synthesis and Innovus P&R	8
5.3.2	FPGA Backend (Zhang): Synthesis and Implementation	13
6	Design Optimizations	13
6.1	ASIC Optimization Journey (Liao)	13
6.2	FPGA Optimization (Zhang)	15
7	Engineering Standards	15
8	Testing / Verification	16
8.1	Verification Flow	16
8.2	ASIC Verification (Liao): Three-Tier + SDF Signoff	17
8.3	FPGA Verification (Zhang): Simulation and On-Board	18
9	Ethical Considerations	18
10	Conclusions	19

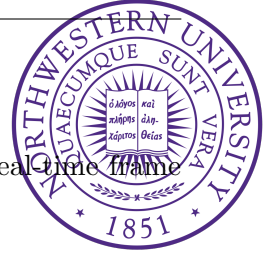


List of Figures

1	MobileFaceNet topology overview.	2
2	ASIC frontend block diagram.	6
3	FPGA accelerator block diagram.	8
4	Final v3 Innovus post-route layout.	10
5	Clock-tree synthesis result.	11
6	Post-route timing and clock-domain reports.	12
7	Routing and area utilisation.	12
8	Innovus post-route power report.	13
9	Post-route floorplan evolution v1 $\rightarrow$ v2 $\rightarrow$ v3.	13
10	Macro placement evolution v2 $\rightarrow$ v3.	14

List of Tables

1	MobileFaceNet Layer Configuration ($3 \times 112 \times 96$ input)	3
2	Precision Comparison Summary	3
3	Fixed-Point Format Selected for Hardware	3
4	Face-Verification Evaluation (C INT16 Q10)	4
5	Similarity Summary: C Fixed-Point vs RTL	4
6	Design Constraints and Requirements	5
7	ASIC vs FPGA Implementation Comparison	6
8	ASIC On-Chip Storage Classification	7
9	4-Stage Activation Pipeline (ASIC)	7
10	Genus Synthesis Results (ASIC, v3)	9
11	Innovus Post-Route Results (ASIC, v3 final)	9
12	ASIC Architecture Evolution: v1 $\rightarrow$ v2 $\rightarrow$ v3	14
13	v3 Cumulative Micro-Optimizations	15
14	Engineering Standards Applied	16
15	Verification Coverage Matrix	17
16	ASIC Verification Results	17
17	Gate-Level Simulation Wall-Clock Comparison	17
18	Abuse Vectors and Mitigations	18
19	Final Metrics vs Design Constraints	19
20	Suggested Follow-on Work	19



Executive Summary

Problem. Edge face verification needs a 50-layer CNN to run under ~ 100 mW at real-time frame rates. General-purpose CPUs and GPUs are too power-hungry to fit that envelope.

Approach. We build a MobileFaceNet-specific convolution accelerator with INT16 Q10 fixed-point arithmetic and run it as a complete 50-layer pipeline along two implementation paths:

- **ASIC path (Liao):** a $12 \times 12 = 144$ -MAC output-stationary systolic array in FreePDK45, with two OpenRAM SRAM macros (activation buffer and partial-sum SRAM) and a 4-stage activation pipeline. Full RTL $\rightarrow$ Genus $\rightarrow$ Innovus flow with SDF-annotated gate-level signoff.
- **FPGA path (Zhang):** a systolic-array CNN accelerator on Zynq FPGA with the same MobileFaceNet workload.

Key Results.

- ASIC: **200 MHz, 14.5 FPS @ $720 \times 720 \mu\text{m}^2$, 96.3 mW, 6.63 μJ /inference**, DRC/connectivity/antenna 0.
- Functional verification: **three-tier 50/50 layer MATCH** (RTL $\equiv$ Genus syn netlist $\equiv$ Innovus post-route netlist), plus zero SDF timing violations.
- FPGA:

Limitations.

- No commercial formal LEC tool (Conformal/Formality) was available; equivalence is shown by full 50-layer gate-level simulation rather than mathematical proof.
- Single-corner timing signoff (typical PVT only); no SI/CRPR analysis.
- ASIC path is design-kit physical implementation only, not taped out.

1 Introduction

Goal. This project implements a fixed-point, MobileFaceNet-specific convolution frontend accelerator. The accelerator runs the full 50-layer MobileFaceNet inference flow autonomously, from the first 3×3 convolution through to the final 7×6 global depthwise convolution and 1×1 linear projection, producing a 128-dimensional face embedding with no host intervention between layers.

Approach overview. Rather than building a fully configurable NPU, the design is *specialized* to MobileFaceNet: INT16 Q10 weights/activations, hard-wired support for the five operator types used in MobileFaceNet (3×3 conv, 3×3 depthwise, 1×1 pointwise, 7×6 global depthwise, residual add), and a 50-entry on-chip ROM that describes the per-layer configuration in 64-bit words. The same baseline RTL is implemented along two independent paths: an ASIC flow in FreePDK45 (12×12 systolic array) and an FPGA flow on Zynq, allowing direct comparison of physical metrics, verification effort, and design productivity between the two technologies.

Prior art. The MobileFaceNet topology itself is taken from Chen et al. [1], with the non-square 112×96 input variant from Sun et al. [4]. Our dataflow design draws on Eyeriss [2] (output-stationary mapping, sliding-window reuse) but specializes the control path to the fixed MobileFaceNet layer sequence. Recent edge-deployment alternatives such as EdgeFace [3] target the same use case at the model level; our work targets the hardware level for an existing trained network.

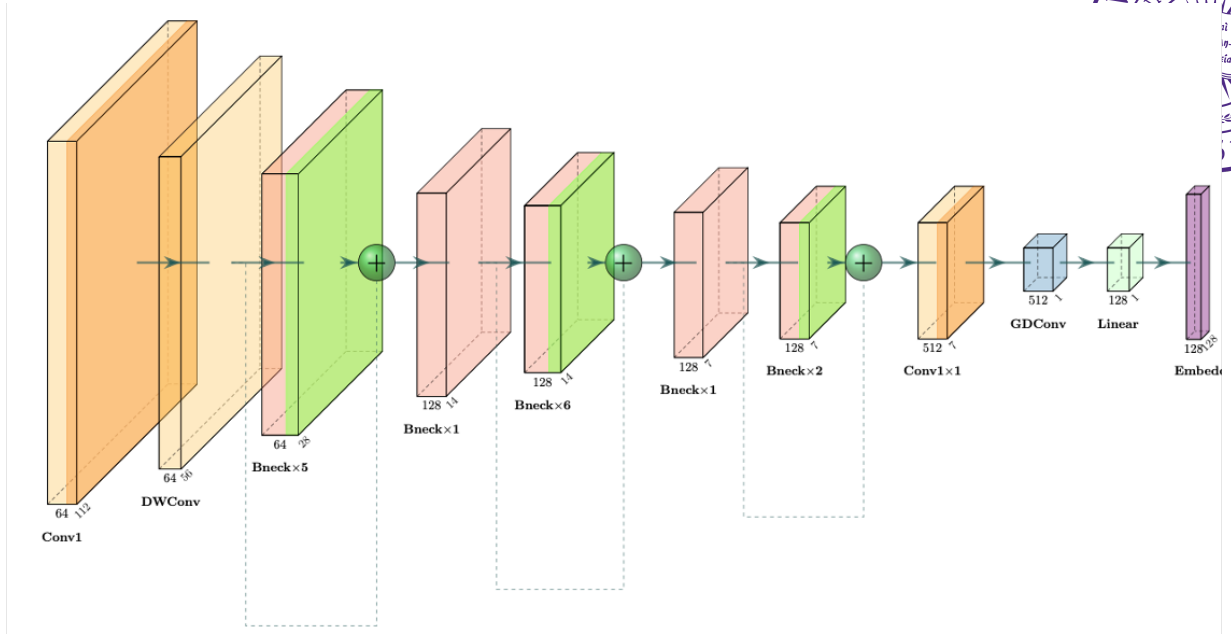
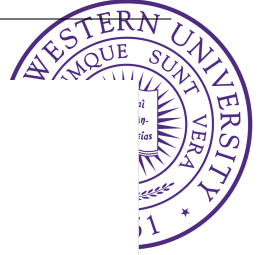


Figure 1: MobileFaceNet topology overview: 50 layers from the initial 3×3 convolution through five bottleneck stages to the final 7×6 global depthwise convolution and 1×1 linear projection producing a 128-d embedding.

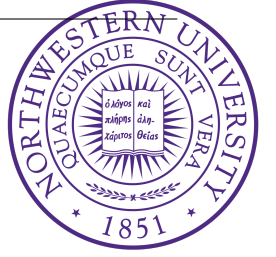
Report structure. Section 3 discusses broader impacts. Section 4 states design constraints. Section 5 describes the hardware architecture (frontend RTL design and backend physical implementation for both ASIC and FPGA). Section 6 enumerates the design optimizations applied. Section 7 lists engineering standards. Section 8 presents functional and timing verification. Section 9 addresses ethical considerations specific to face recognition. Section 10 concludes.

2 Model and Numerical Design

Before any hardware is built, the model topology and the numerical format must be fixed. Both decisions are constraints that the hardware datapath is then specialised around.

2.1 MobileFaceNet Topology

We target the MobileFaceNet variant with input size $3 \times 112 \times 96$. The original MobileFaceNet paper [1] and the Eyeriss-style accelerator in [2] use a square $3 \times 112 \times 112$ input with a 7×7 global depthwise convolution. We instead adopt the non-square variant from [4], which replaces the GDConv kernel with 7×6 to match the reduced horizontal resolution. The complete layer schedule is shown in Table 1. When each Bottleneck is expanded into its three constituent sub-layers (PW1, DW, PW2), the full model comprises **50 layers** (L0–L49).

Table 1: MobileFaceNet Layer Configuration ($3 \times 112 \times 96$ input)

Input shape	Operator	t	c	n	s
$3 \times 112 \times 96$	Conv 3×3	–	64	1	2
$64 \times 56 \times 48$	DW Conv 3×3	–	64	1	1
$64 \times 56 \times 48$	Bottleneck	2	64	5	2
$64 \times 28 \times 24$	Bottleneck	4	128	1	2
$128 \times 14 \times 12$	Bottleneck	2	128	6	1
$128 \times 14 \times 12$	Bottleneck	4	128	1	2
$128 \times 7 \times 6$	Bottleneck	2	128	2	1
$128 \times 7 \times 6$	Conv 1×1	–	512	1	1
$512 \times 7 \times 6$	GDCConv 7×6	–	512	1	1
$512 \times 1 \times 1$	Linear (PW 1×1)	–	128	1	1

2.2 Precision Selection

A face-verification network must preserve its embedding’s discriminative property under quantisation: the gap between intra-person and inter-person cosine similarity must remain large. We evaluated four precision formats (PyTorch float32, a plain-C float32 reference, C INT16 Q10, and C INT8 Q4) by computing pairwise cosine similarities on a face-verification task and comparing the largest absolute error against the PyTorch reference. Results are in Table 2.

Table 2: Precision Comparison Summary

Format	Max error vs PyTorch	Verdict
C float32	≈ 0	Reference
C INT16	< 0.003 (most pairs: OK)	Selected
C INT8	up to 0.34 (many pairs: WARN)	Rejected

The C float32 path closely matches the PyTorch reference, confirming the floating-point implementation. C INT16 (Q10 fixed-point) introduces only negligible error on the majority of pairs, while C INT8 has significantly larger deviations, especially on pairs whose similarity is already near zero (the regime that decides match/no-match). We therefore select **INT16 Q10** as the hardware representation: best trade-off between hardware cost and numerical accuracy for this workload.

Table 3 fixes the bit-widths used downstream by all hardware modules.

Table 3: Fixed-Point Format Selected for Hardware

Quantity	Format	Width
Activation	Q10	int16
Weight	Q10	int16
Bias	Q20	int32
Accumulator	Q20	int40
PReLU α	Q10	int16

2.3 Model Evaluation

To validate the selected INT16 Q10 model end-to-end, intra-person and inter-person cosine similarities were measured on two datasets: a 6-person / 3-images-each public set and a self-prepared set containing four well-known individuals (Dan, Elon, Trump, Wil). Results in Table 4 show a large separability gap (intra $\gg$ inter) on both datasets, confirming that INT16 Q10 quantisation preserves the model’s discriminative ability with negligible loss against the floating-point reference.



Table 4: Face-Verification Evaluation (C INT16 Q10)

Dataset	Intra avg	Inter avg	Gap
Original (6 persons, 3 images each)	0.5575	0.0673	+0.4903
Self-prepared (Dan, Elon, Trump, Wil)	0.5270	0.0783	+0.4487

Finally, Table 5 compares the C fixed-point reference against the RTL output on the self-prepared dataset. The two are bit-identical, confirming that the hardware datapath faithfully implements the chosen fixed-point arithmetic.

Table 5: Similarity Summary: C Fixed-Point vs RTL

Metric	C fixed	RTL
Intra avg	0.5270	0.5270
Inter avg	0.0783	0.0783
Gap	+0.4487	+0.4487

This bit-exactness against the C-model golden is the foundation for all subsequent verification at the RTL, synthesised, and post-route levels (Section 8).

3 Broader Considerations

Face verification is one of the most widely deployed biometric modalities, shipping in hundreds of millions of units a year through smartphones, smart locks, doorbell cameras, point-of-sale terminals, and access gates. Almost all of those products use edge inference: the embedding network runs on the device, and only the 128-dimensional vector or a match decision leaves the chip. An accelerator that fits the sub-100 mW envelope of a battery-powered device at real-time frame rates is a direct commercial fit for that segment.

The clearest integration target is as a small NPU sub-system inside a larger SoC, sharing the host AXI bus and memory hierarchy. Concrete uses we considered were mobile face unlock with an always-on camera, smart doorbells and cameras that recognise locally instead of streaming to the cloud, badge-less building access running offline, and in-vehicle driver identification where cloud round-trip latency is unacceptable.

Pushing face recognition to the edge has both privacy benefits and risks. On the positive side, raw images and templates never leave the device, so there is no centralised database for attackers or governments to compel. On the negative side, the same drop in cost and power makes mass deployment in surveillance contexts much easier; the same chip that unlocks a private user’s phone can identify a stranger on a public street. The technology is dual-use, and deployment policy decides which outcome dominates. Specific safeguards are in Section 9.

Several legal frameworks already constrain how this kind of accelerator can be productised. Face data is special-category personal data under EU GDPR Article 9, with similar provisions in California’s CCPA and other jurisdictions. The EU AI Act (Articles 5 and 6) classifies real-time remote biometric identification in public spaces as either prohibited or high-risk depending on context. A commercial product built on this accelerator therefore needs consent capture, purpose limitation, on-device template storage, and a documented opt-out path. These obligations shape the chip interface in concrete ways: only the 128-d embedding is exposed off-die, the raw image stays inside, and in a production SoC the embedding storage would sit inside a hardware-isolated enclave.

4 Design Constraints and Requirements

Table 6 summarises the hard and soft constraints that drove the architectural and implementation decisions.

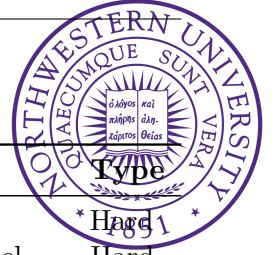


Table 6: Design Constraints and Requirements

Category	Constraint / Requirement	Type
Functional	Execute 50 MobileFaceNet layers autonomously	Hard
Functional	Produce 128-d float-equivalent embedding within <0.001 of C-model	Hard
Numerics	INT16 Q10 weights/activations, Q20 accumulators	Hard
Process (ASIC)	FreePDK45 standard-cell library (NCSU PDK)	Hard
Memory IP	No commercial SRAM IP; must use OpenRAM open-source generator	Hard
Clock target	≥ 100 MHz baseline; 200 MHz stretch goal	Soft
Power budget	< 100 mW (edge battery-powered use case)	Soft
Area target	$< 1 \text{ mm}^2$ core area (educational-PDK norm)	Soft
DRC / antenna	Zero violations at signoff	Hard
Timing signoff	Setup/hold WNS > 0 at all corners run	Hard
Verification	50/50 layer bit-exact match against C-model	Hard
Toolchain	Cadence Genus (synthesis), Innovus (P&R), Xcelium (simulation)	Hard
FPGA target		Hard

4.1 Process and IP Availability

The FreePDK45 educational PDK provides only standard-cell timing and LEF/Liberty data; no foundry SRAM compiler is included. We therefore generated all on-chip SRAM macros via the OpenRAM open-source SRAM compiler, which produces synthesizable behavioural models for simulation and LEF/Liberty/GDS views for physical implementation. This constraint shaped the memory subsystem: we use only two SRAM macros (activation buffer and partial-sum buffer) and keep small constant storage (weights, biases, PReLU α , layer configuration) as combinational ROMs that the synthesizer maps to standard-cell mux trees.

4.2 Verification Capability

The lab environment does not provide a commercial formal-equivalence checker (Conformal LEC / Formality), nor signal-integrity-aware static timing (PrimeTime SI). We therefore replace formal LEC with full 50-layer gate-level simulation against the C-model golden vectors, and replace SI timing with SDF-annotated gate-level simulation using the Innovus-extracted SDF. These substitutions are slower than the commercial flows (a single full SDF run takes several days) but provide functionally equivalent coverage for non-tape-out signoff.

5 Hardware Architecture

5.1 Architecture Overview

Both implementations share the same algorithmic specification (MobileFaceNet INT16 Q10, 50-layer sequential execution, residual shortcuts, mixed standard / depthwise / pointwise convolutions) but realise the compute and memory subsystems differently to suit the target technology. Table 7 contrasts the two paths.

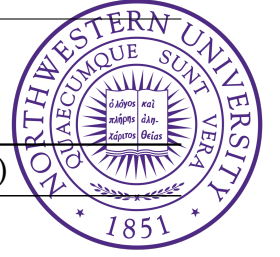


Table 7: ASIC vs FPGA Implementation Comparison

	ASIC path (Liao)	FPGA path (Zhang)
Technology	FreePDK45 (45 nm)	
Compute fabric	12×12 = 144-MAC output-stationary systolic array	
Activation memory	OpenRAM SRAM macro (192-bit × 64, 1RW)	
Partial-sum memory	OpenRAM SRAM macro (40-bit × 512, 1RW+1R)	
Weight storage	Combinational ROM (synthesised mux tree)	
Target clock	5.0 ns (200 MHz)	
Achieved metric	14.5 FPS, 96.3 mW	
Verification	RTL + Genus syn + Innovus post-route + SDF	
Implementation tools	Cadence Genus + Innovus	

The two implementations are complementary: the ASIC path quantifies the lower bound on power and the upper bound on frequency achievable in 45 nm silicon, while the FPGA path provides on-board functional validation and rapid bring-up.

5.2 Frontend Design (RTL Architecture)

The frontend design covers the register-transfer-level architecture shared by both implementation paths: the compute datapath, the memory hierarchy at the RTL level, and the controller finite-state machine. ASIC-specific and FPGA-specific RTL details follow in the two sub-sub-sections below.

5.2.1 ASIC Frontend (Liao)

The ASIC RTL is organised into eight modules under `mfn_frontend_top_v3`. Figure 2 shows the block-level dataflow.

Block diagram placeholder — export doc/v3_architecture.html to PNG.

Figure 2: ASIC frontend block diagram. Dataflow: pixel input → act_buf SRAM → 12×12 systolic array → psum SRAM → activation → next-layer feature map.

Compute datapath. The core compute unit is `mfn_sa_12x12`, a $12 \times 12 = 144$ processing-element (PE) output-stationary systolic array. Each PE `mfn_pe` holds one output channel’s 40-bit accumulator and performs a single signed $16 \times 16 \rightarrow 32$ -bit multiply followed by a sign-extended add per cycle. The array supports two dataflow modes selected by `dw_mode`:

- **Pointwise / standard mode** (`dw_mode=0`): each row computes the 12-tap dot product $\sum_{c=0}^{11} \text{act_in}[c] \cdot \text{wgt_in}[r][c]$. All 144 MACs fire per cycle, parallel across 12 output channels and 12 input channels.
- **Depthwise mode** (`dw_mode=1`): each row uses its own activation lane (diagonal mapping) since depthwise convolutions never mix channels. The array degrades to 12 independent MACs per cycle, mirroring the natural parallelism of the operation.

A pipeline register (`row_dot_reg`, 12×40-bit) between the multiplier-adder tree and the final accumulator update splits the critical path, enabling the 5.0 ns / 200 MHz target.



Memory hierarchy. Three classes of on-chip storage are used:

Table 8: ASIC On-Chip Storage Classification

Storage	Implementation	Size	Rationale
Activation buffer	OpenRAM SRAM (1RW)	$192\text{ b} \times 64$	12 channels $\times$ 16-bit per word matches SA column width
Partial-sum buffer	OpenRAM SRAM (1RW+1R)	$40\text{ b} \times 512$	holds up to 512 output-channel partial sums (linear1)
Weight ROM	Combinational logic	$\sim 1\text{ M} \times 16\text{-bit}$	content fixed at synthesis; 12 read ports for SA row
Bias / PReLU ROM	Combinational logic	per-channel scalars	small, value-constant
Layer configuration ROM	Combinational logic	$50 \times 64\text{-bit}$	one 64-bit descriptor per layer

The choice between SRAM and ROM is driven by access pattern and size: large random-access read/write storage uses SRAM; small read-only constants synthesise more cheaply as gates.

4-stage activation pipeline. To meet 200 MHz the path from address generation to PE accumulator update is split into four pipeline stages, summarised in Table 9.

Table 9: 4-Stage Activation Pipeline (ASIC)

Stage	Function	Implementation
Stage 0	Address generation	controller drives $(x, y, c_{in}) \rightarrow \text{act_buf read addr}$
Stage 1	SRAM access (1-cycle latency)	OpenRAM macro internal read
Stage 2	Lane select + zero-pad	<code>act_out_reg</code> latches dout on negedge
Stage 3	SA dot-product register	<code>row_dot_reg</code> (12 $\times$ 40-bit)

Controller FSM. `mfn_controller_v3` is a 25-state machine partitioned into three sub-paths, one per convolution mode:

- Pointwise / standard path (`S_ACT_LOAD` $\rightarrow$ `S_BIAS_LOAD` $\rightarrow$ `S_CIN_COMPUTE` $\rightarrow$ `S_WRITEBACK`),
- Depthwise path (`S_DW_FETCH` $\rightarrow$ `S_DW_COMPUTE` $\rightarrow$ `S_DW_DRAIN` $\rightarrow$ `S_DW_WRITEBACK`),
- Depthwise-12 fast path for layers whose channel count is a multiple of 12 (`S_DW12_BIAS` $\rightarrow$ `S_DW12_COMPUTE` $\rightarrow$ `S_DW12_KP_NEXT`).

All address generation, SA enable/clear, and SRAM write enables are driven directly from the FSM; there is no host intervention between layers.

5.2.2 FPGA Frontend (Zhang)

The FPGA path implements the same MobileFaceNet workload using a distinct microarchitecture suited to LUT/BRAM/DSP resources on Zynq. The design is organised as a closed-loop accelerator: global SRAM buffers feed a switchable layer device, whose outputs are routed through a shared post-process unit and a write-back path that returns activations to SRAM for the next layer.

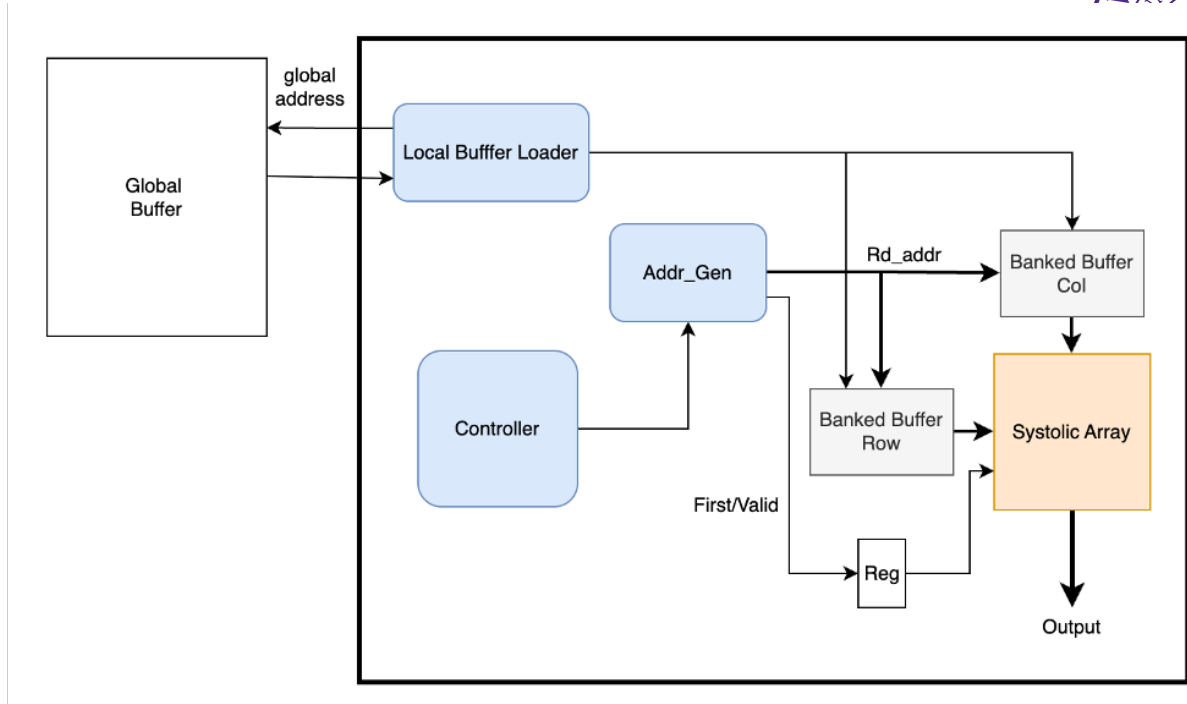
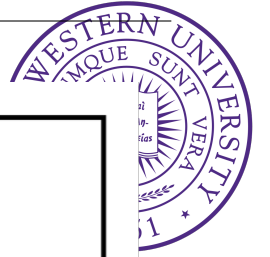


Figure 3: System-level architecture of the closed-loop FPGA systolic-array accelerator.

The major sub-blocks visible in Figure 3 are:

1. **A/O SRAM**: stores input activations and the intermediate/final layer outputs.
2. **Weight SRAM**: stores quantised weights for all supported layers.
3. **Switcher**: selects the active layer device and routes SRAM read/write channels.
4. **Layer device**: contains the systolic-array datapath for the operator currently being executed (preload, SA compute, residual read, tile-level control).
5. **Post-process**: converts the wide partial sums to 8-bit activations and applies bias, requantisation, residual add, and PReLU as required.
6. **Write-back buffer / controller**: packs processed outputs into SRAM words and generates the next-layer write addresses.

The system was first brought up with a $\text{conv}1 \times 1$ layer device, which exercises every shared resource (switcher, post-process, write-back path) end to end. The decoupled interface means subsequent layer devices (depthwise, 3×3 standard convolution) can reuse the switcher, post-process, and write-back path without modification.

Compute datapath. The layer device's SA performs 1×1 pointwise multiplications with a tile-granular preload + compute schedule.

Memory subsystem.

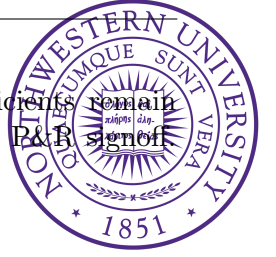
Host interface.

5.3 Backend Design (Physical Implementation)

The backend design covers the technology-specific mapping of the RTL to a physical implementation: standard-cell synthesis and place-and-route for the ASIC path, and FPGA synthesis and bitstream generation for the FPGA path.

5.3.1 ASIC Backend (Liao): Genus Synthesis and Innovus P&R

The v3 RTL is synthesised with Cadence Genus against the FreePDK45 standard-cell library. The two OpenRAM SRAMs (`sram_act_buf_1rw0r0w_192_64` and `sram_psum_a_1rw1r0w_40_512`) are treated



as black boxes via their Liberty and LEF views; weights, biases, and PReLU coefficients remain combinational ROMs. Synthesis runs at a 5.0 ns target period to align with the final P&R signoff. Results are in Table 10.

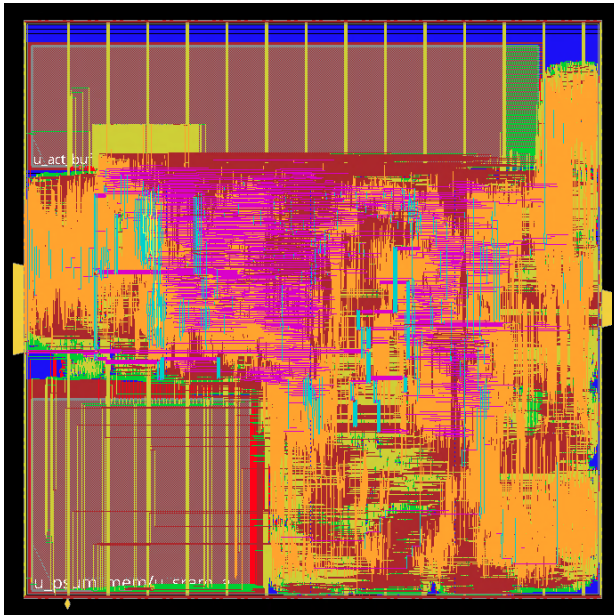
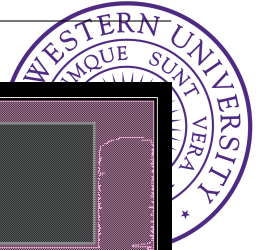
Table 10: Genus Synthesis Results (ASIC, v3)

Metric	Value
Target clock period	5.0 ns (200 MHz)
Worst slack (WNS)	+0.177 ns
Total cells	88,018
Total area (cells only)	334,956 μm^2
SRAM macros	2 (act_buf 97 K μm^2 + psum_a 66 K μm^2)
Critical path	u_act_buf/act_out_reg $\rightarrow$ u_sa/mul_55_31 (16 $\times$ 16) $\rightarrow$ CSA tree $\rightarrow$ u_sa/acc_reg[0][39]

The netlist is then placed and routed in Cadence Innovus (`run_innovus_3.tcl`) on a 720 $\times$ 720 μm^2 floorplan, shrunk from an initial 800 $\times$ 700 once timing closed. The two SRAM macros are placed manually so the SA dataflow stays compact, and the clock tree uses non-default routing (CTS_2W1S on leaf nets, CTS_2W2S on trunks with VSS shielding) to hold skew under 70 ps. Before LEF import, `fix_sram_lef.py` strips the over-conservative M3/M4 obstructions from the OpenRAM LEF and snaps off-grid pin coordinates to the 0.005 μm manufacturing grid; without this step the SRAM dout pins on M3 have no legal via3/M4 escape and the router produces $\sim$ 480 DRC SHORT/SPACING violations. The final post-route metrics are in Table 11 and the layout in Figure 4.

Table 11: Innovus Post-Route Results (ASIC, v3 final)

Category	Result
Core floorplan	720 $\times$ 720 μm^2 (518,400 μm^2)
Placement density	59.5 %
Setup WNS	+0.089 ns (MET)
Hold WNS	+0.437 ns (MET)
Setup / Hold TNS	0 / 0
DRC violations	0
Connectivity	no problems or warnings
Antenna	no violations
Total power	96.32 mW (Switching 43.76, Internal 24.6, Leakage 3.0, Macro 24.9)
Cycles per inference	13,773,428
Throughput	14.52 FPS @ 200 MHz on 112 $\times$ 96 input
Energy per inference	6.63 μJ



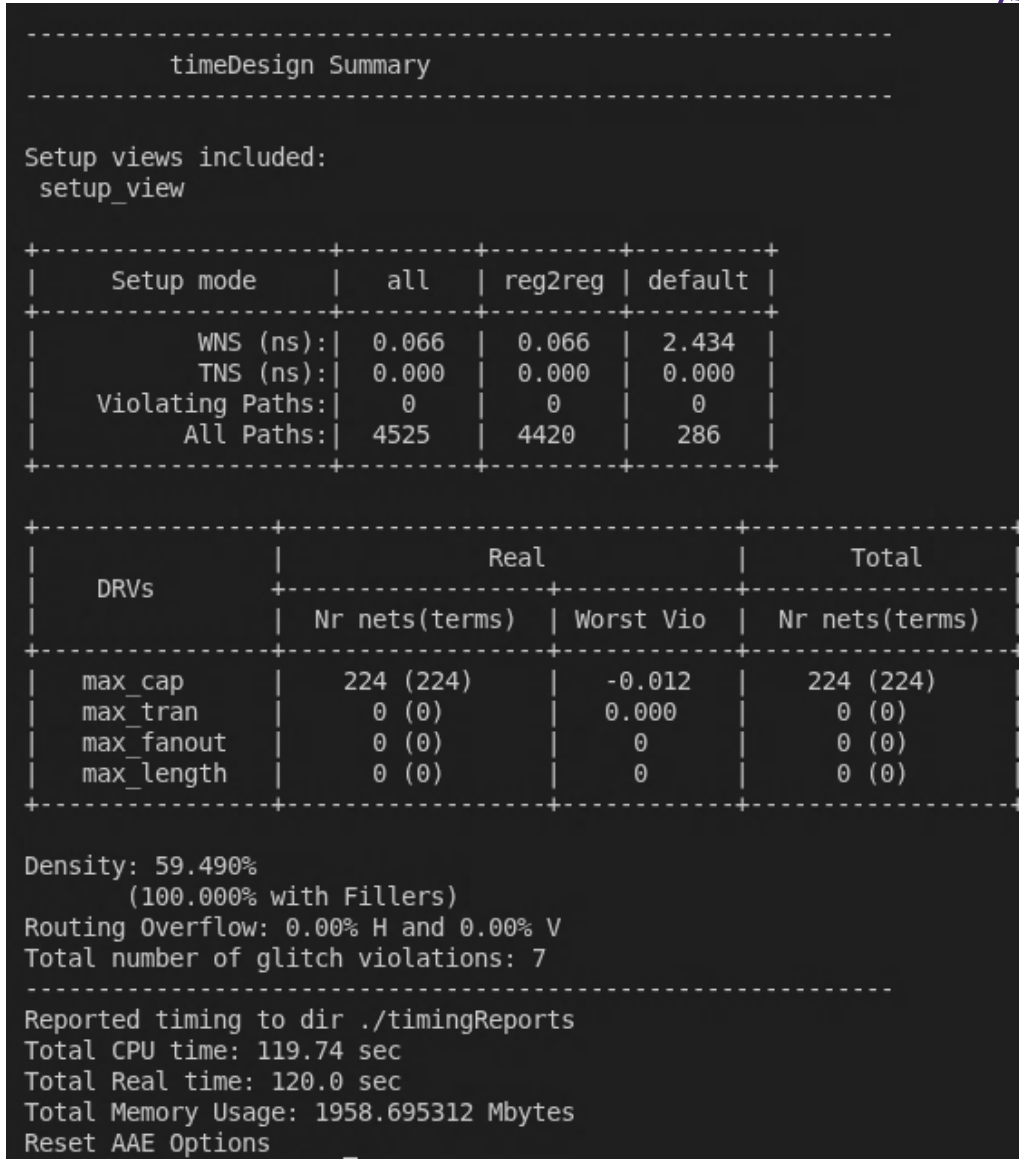
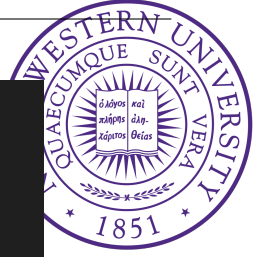
(a) Full die view ($720 \times 720 \mu\text{m}^2$).



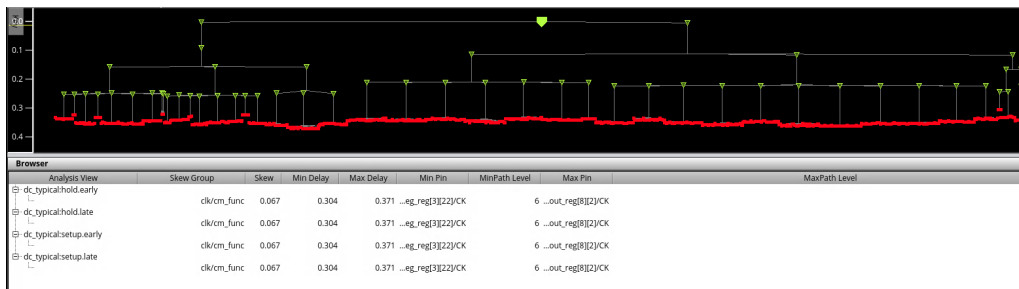
(b) Detail view: SA + SRAM macros + controller cells.

Figure 4: Final v3 Innovus post-route layout. Two OpenRAM SRAM macros (activation buffer left, partial-sum buffer right) bracket the central 144-MAC systolic-array placement.

The CTS report (Figure 5) shows balanced clock arrival times across the die with insertion-delay $\mu = 0.348 \text{ ns}$, $\sigma = 0.009 \text{ ns}$ and skew 0.067 ns .



(a) Post-CTS placement and clock buffers.



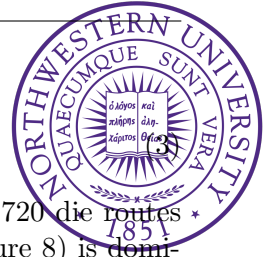
(b) Clock-tree topology and balanced fanout.

Figure 5: Clock-tree synthesis result. Skew 0.067 ns, insertion-delay $\mu = 0.348$ ns, $\sigma = 0.009$ ns.

With WNS = +0.089 ns on the 5.0 ns target, the critical path measures $T_{\text{crit}} = 4.911$ ns. The achievable frequency and the resulting throughput and energy are then

$$F_{\text{max}} = \frac{1}{T_{\text{crit}}} = \frac{1}{4.911 \text{ ns}} \approx 203.6 \text{ MHz}, \quad (1)$$

$$\text{FPS} = \frac{F_{\text{clk}}}{C_{\text{inf}}} = \frac{200,000,000}{13,773,428} \approx 14.52 \text{ frames/s}, \quad (2)$$



$$T_{\text{inf}} = \frac{C_{\text{inf}}}{F_{\text{clk}}} \approx 68.87 \text{ ms}, \quad E_{\text{inf}} = P_{\text{total}} \cdot T_{\text{inf}} \approx 6.63 \mu\text{J}.$$

The Innovus reports (Figures 6 and 7) confirm these numbers and show the 720×720 die routes without congestion hotspots at 59.5% placement density. The power breakdown (Figure 8) is dominated by switching activity in the 144-MAC SA, as expected when 144 multipliers toggle every cycle at 200 MHz.

```
innovus 6> report_timing
#####
# Generated by: Cadence Innovus 16.20-p002.1
# OS: Linux x86_64 (Host ID moore.wot.ece.northwestern.edu)
# Generated on: Tue May 26 23:10:42 2026
# Design: mfn_frontend_top_v3
# Command: report_timing
#####
Path 1: MET Setup Check with Pin u_sa/row_dot_reg_reg[8][39]/CK
Endpoint: u_sa/row_dot_reg_reg[8][39]/SI (^) checked with leading edge of
'clk'
Beginpoint: u_act_buf/act_out_reg[0][0]/Q (^) triggered by leading edge of
'clk'
Path Groups: {clk}
Analysis View: setup_view
Other End Arrival Time 0.427
- Setup 0.066
+ Phase Shift 5.000
- Uncertainty 0.100
= Required Time 5.262
- Arrival Time 5.172
= Slack Time 0.089
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) -0.033
= Beginpoint Arrival Time -0.033
```

(a) Innovus report_timing: setup WNS +0.089 ns.

```
innovus 7> report_clocks
#####
# Generated by: Cadence Innovus 16.20-p002.1
# OS: Linux x86_64 (Host ID moore.wot.ece.northwestern.edu)
# Generated on: Tue May 26 23:10:48 2026
# Design: mfn_frontend_top_v3
# Command: report_clocks
#####
+-----+-----+
| Clock Descriptions | Attributes |
+-----+-----+
| Clock | Source | View | Period | Lead | Trail | Generated | Propagated |
| Name | | | | | | | |
+-----+-----+
| clk | clk | setup_view | 5.000 | 0.000 | 2.500 | n | n |
+-----+-----+
```

(b) Per-clock summary: 200 MHz domain.

Figure 6: Post-route timing and clock-domain reports.

```
-----
timeDesign Summary
-----
Setup views included:
setup_view

+-----+-----+-----+-----+
| Setup mode | all | reg2reg | default |
+-----+-----+-----+-----+
| WNS (ns): | 0.089 | 0.089 | 2.394 |
| TNS (ns): | 0.000 | 0.000 | 0.000 |
| Violating Paths: | 0 | 0 | 0 |
| All Paths: | 4525 | 4420 | 286 |
+-----+-----+-----+-----+

+-----+-----+-----+-----+
| DRVs | Real | Total |
| Nr nets (terms) | Worst Vio | Nr nets (terms) |
+-----+-----+-----+-----+
| max_cap | 222 (222) | -0.012 | 222 (222) |
| max_tran | 0 (0) | 0.000 | 0 (0) |
| max_fanout | 0 (0) | 0 | 0 (0) |
| max_length | 0 (0) | 0 | 0 (0) |
+-----+-----+-----+-----+

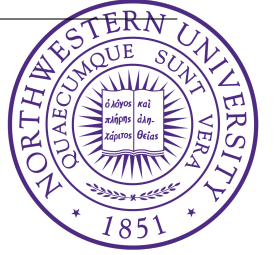
Density: 59.490%
(100.000% with Fillers)
Total number of glitch violations: 7
-----
Reported timing to dir ./timingReports
Total CPU time: 74.91 sec
Total Real time: 75.0 sec
Total Memory Usage: 2024.429688 Mbytes
Reset AAE Options
```

(a) Post-route view (optsRoute).

```
innovus 4> report_area
Depth Name #Inst Area (um^2)
-----
0 mfn_frontend_top_v3 90360 359740.49105
1 u_psum_mem 85 66448.993
1 u_act 812 2249.296
1 u_sa 79630 173958.68
1 u_ctrl 7994 16799.496
1 u_bias_rom 0 0.0
1 u_act_buf 1642 100058.72405
1 u_wgt_rom 0 0.0
1 u_prelu_rom 0 0.0
1 u_cfg_rom 0 0.0
1
```

(b) Area / utilisation report.

Figure 7: Routing and area utilisation: zero DRC violations, 59.5% density.



Total Power		

Total Internal Power:	48.60088682	50.4594%
Total Switching Power:	43.75658635	45.4298%
Total Leakage Power:	3.95939664	4.1108%
Total Power:	96.31686973	

Figure 8: Innovus post-route power report: total 96.32 mW, dominated by switching power in the 144-MAC SA at 200 MHz.

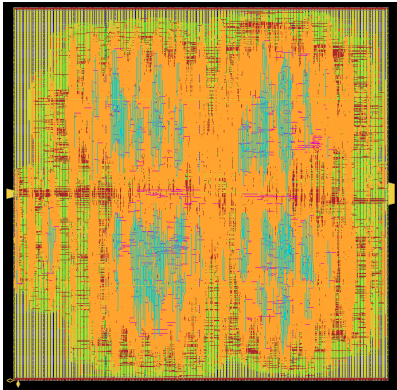
5.3.2 FPGA Backend (Zhang): Synthesis and Implementation

Placeholder — Zhang’s FPGA backend results go here. Should mirror the ASIC backend (Section 5.3.1): toolchain, target device, resource utilisation, timing closure result, power estimate, and on-board measurement.

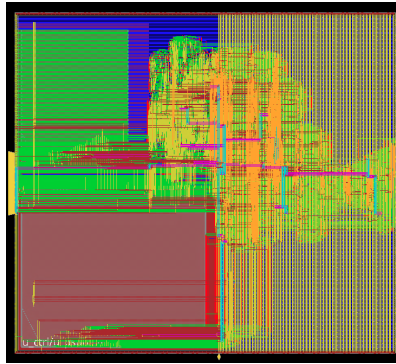
6 Design Optimizations

6.1 ASIC Optimization Journey (Liao)

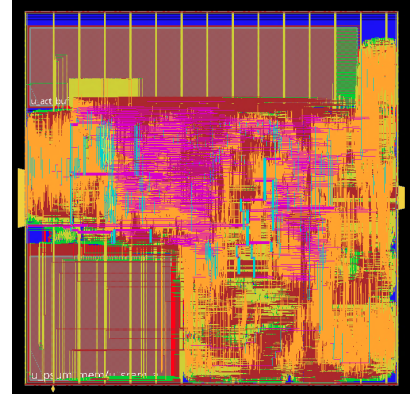
The ASIC implementation went through three architectural generations and four micro-architectural rounds. Each iteration was driven by a measurable shortfall in throughput, power, or area observed at the prior signoff. Figure 9 shows the post-route floorplan for each generation; Table 12 quantifies the metrics behind them.



(a) v1: 1×9-MAC.

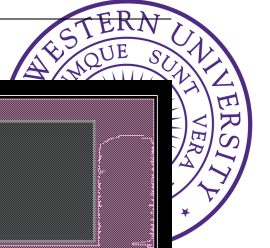


(b) v2: 2×9-MAC + psum SRAM.



(c) v3: 12×12 SA + 2 SRAMs.

Figure 9: ASIC architecture evolution: post-route floorplans for v1, v2, and v3. Visible progression from a single MAC datapath (v1) to a dual-MAC datapath with one OpenRAM macro (v2) to a 12×12 systolic array with two OpenRAM macros (v3, the final design).



(a) v2 detail: psum SRAM macro placed bottom-left.



(b) v3 detail: act_buf and psum SRAM macros bracket the SA.

Figure 10: Macro placement evolution v2 $\rightarrow$ v3.Table 12: ASIC Architecture Evolution: v1 $\rightarrow$ v2 $\rightarrow$ v3

	v1			v2		v3 (final)
Compute	single	9-MAC	dot-	dual 9-MAC (18 mult, 2 out-ch parallel)		$12 \times 12 = 144$ -MAC SA
Adder	38-stage ripple carry			40-bit 2-level CLA		CLA + Stage 3 SA pipeline
psum memory	512 $\times$ 40 FF			+ OpenRAM SRAM macro (psum)		+ OpenRAM SRAM macro (1RW+1R)
act memory	FF array			FF array		+ OpenRAM SRAM macro (192 $\times$ 64)
Cycles / inf	43.16 M			25.77 M		13.77 M
Clock	100 MHz			125 MHz		200 MHz
FPS	2.3			4.85		14.5
Power	26.7 mW			11.4 mW		96.3 mW
Core area	~ 285 K μm^2			479 K μm^2		518 K μm^2
DRC	0			0		0

v1's single 9-MAC unit produced one output channel per cycle, so 8/9 of the available parallelism was wasted on pointwise layers. v2 doubled the MAC count to 18 (two parallel output channels) and replaced the 38-stage ripple-carry adder with a 40-bit two-level CLA. The biggest win was moving the 512 $\times$ 40 psum memory from an FF array into an OpenRAM SRAM macro: total cell count dropped and post-route power fell from 26.7 mW to 11.4 mW even though the MAC count had doubled.

v3 then generalised to a $12 \times 12 = 144$ -MAC output-stationary systolic array (`mfn_sa_12x12`), each PE holding a 40-bit accumulator. The cycle count roughly halved again (25.77 M to 13.77 M). Power rises to 96.3 mW because 144 multipliers now toggle every cycle at 200 MHz; the meaningful figure of merit is energy per inference, which lands at 6.63 μJ .

Within v3, four cumulative micro-optimizations were applied (Table 13). Each was driven by the previous round's WNS report or power breakdown.



Table 13: v3 Cumulative Micro-Optimizations

Round	Change	F_{max}	FPS	Power
Baseline (2-stage activation)	Stage 0+1 only	87 MHz	6.0	45.4 mW
+ Stage 0+2 split	Lane-select pipelined	150 MHz	10.9	77.9 mW
+ Stage 3 SA <code>row_dot_reg</code>	<code>acc + row_dot</code> split off	174 MHz	12.6	88.2 mW
+ 5.0 ns push	T_{clk} tightened only	200 MHz	14.5	99.1 mW
+ 720×720 area shrink	smaller die, denser placement	200 MHz	14.5	96.3 mW

Two non-architectural fixes were also required to reach signoff. The first is a LEF patch: the OpenRAM-emitted SRAM LEF reserves the whole of metal-3 and metal-4 as obstructions on every macro, but the read-data pins sit on metal-3, so the router has no legal via-3 escape and is forced to cross the obstructions. Without the patch the design reports ~480 SHORT/SPACING DRC violations. The script `fix_sram_lef.py` removes the M3/M4 OBS blocks (the OpenRAM behavioural macros do not actually route on those layers) and snaps off-grid RECT and SIZE coordinates to the 0.005 μm manufacturing grid. After patching, P&R is DRC-clean.

The second is a testbench race. The first 50-layer post-route gate-level simulation reported all 50 layers as mismatches even though RTL was 50/50 MATCH. A `bind`-based probe localised the cause to the testbench: `pixel_in` was driven on `@(posedge clk)` from the address the DUT presented on `x_out`, but the gate-level netlist adds delta-cycle delay on port outputs, so the testbench latched the previous-cycle address. Driving on `@(negedge clk)`, after `x_out` has settled, restored all 50 layers to exact match.

6.2 FPGA Optimization (Zhang)

Placeholder — Zhang’s FPGA optimization narrative goes here, mirroring the structure of Section 6.1: quantitative “before $\rightarrow$ after” for each optimization (F_{max} , resource use, throughput).

7 Engineering Standards

Table 14 lists the engineering standards and reference specifications used throughout the design.



Table 14: Engineering Standards Applied

Domain	Standard	Where used
HDL	IEEE 1800-2017 (SystemVerilog)	All RTL modules (synthesisable subset)
HDL	IEEE 1364-2005 (Verilog 2005)	OpenRAM-emitted SRAM behavioural models
Library exchange	LEF/DEF (Si2 OpenAccess)	Standard-cell + SRAM physical abstracts; Innovus floorplan import/export
Timing models	Liberty (.lib, Synopsys-de-facto)	Cell delays for Genus / Innovus / Xcelium
Delay annotation	IEEE 1497 (SDF)	Post-route SDF backannotation for gate-level signoff
Constraints	Synopsys SDC	<code>constraints.v3.sdc</code> : clock, IO delay, false paths
Reference floats	IEEE 754 binary32	PyTorch baseline + C-model fp32 reference
Fixed-point	Q10 (data/weights), Q20 (accumulator)	internal datapath
Process design kit	NCSU FreePDK45	ASIC standard-cell library + DRC rules
Memory IP	OpenRAM (open-source)	192×64 act_buf, 512×40 psum SRAMs

Compliance notes. The RTL is written in the synthesisable subset of SystemVerilog 2017 (no `assert`, no *dynamic*, no `class`, limited use of `interface`). All cell timing checks (setup/hold/recovery/removal) follow the standard SDF semantics. The LEF patching script `fix_sram_lef.py` preserves the LEF syntax conventions specified in the Si2 LEF/DEF Language Reference; no extension keywords are introduced.

8 Testing / Verification

8.1 Verification Flow

A single golden reference is used to verify both implementation paths:

```

PyTorch (float32) → C-model (INT16 Q10)
                  → per-layer golden hex vectors
                  → implementation under test
                  → bit-level diff

```

The C-model is bit-exact against the chosen Q10/Q20 fixed-point arithmetic and is the canonical reference: any RTL, gate-level, or FPGA output is compared against it layer by layer. End-to-end face verification (intra/inter-person cosine similarity gap) is also measured to confirm the 128-d embedding quality is preserved.

Coverage. Verification coverage is defined at three levels:

- **Functional:** each of the 50 MobileFaceNet layers must produce an output that matches the C-model golden hex file exactly (zero bits different across all output pixels).
- **End-to-end:** the 128-d embedding vector for a set of face images must match the C-model embedding bit-exactly.
- **Timing** (ASIC only): the post-route netlist with SDF backannotation must complete the same simulation with no setup/hold timing-check violations.



Table 15 shows the verification matrix for both paths.

Table 15: Verification Coverage Matrix

Level	ASIC	FPGA	Method
RTL functional (50-layer)	✓	✓	Xcelium / Vivado simulator vs C-model
Synthesised netlist (gate-level)	✓	–	Xcelium GL vs C-model golden
Post-route netlist (gate-level)	✓	–	Xcelium GL on Innovus netlist vs C-model
SDF-annotated (timing)	✓	–	Xcelium GL + SDF, check timing violations
On-board	–	✓	Zynq board capture + AXI-Stream readback
End-to-end embedding	✓	✓	128-d cosine vs C-model

8.2 ASIC Verification (Liao): Three-Tier + SDF Signoff

The ASIC path is verified at four progressively stricter levels; results are summarised in Table 16.

Table 16: ASIC Verification Results

Level	Stimulus	Delay model	Result
L1	RTL	zero-delay	50 / 50 MATCH
L2	Genus syn netlist	zero-delay	50 / 50 MATCH
L3	Innovus post-route	zero-delay	50 / 50 MATCH
L4	Innovus post-route	SDF annotated	50 / 50 MATCH, 0 violations
End-to-end embedding (RTL, 13 face images): 128 / 128 exact, MaxDiff = 0			

Levels L1–L3 collectively establish that the design behaves identically at the RTL, synthesised, and post-route stages. A formal Logic Equivalence Checker (Conformal LEC, Formality) would normally produce this result in seconds; without one available we substitute a full 50-layer gate-level simulation against the C-model golden hex vectors. The trade-off is wall-clock time. Table 17 summarises how long each of the three gate-level variants took on the same machine.

Table 17: Gate-Level Simulation Wall-Clock Comparison

Run	Netlist	Delay model	Layers done	Wall-clock
<code>make gl</code>	Genus syn	zero-delay	50 / 50	~78 h
<code>make pnr</code>	Innovus post-route	zero-delay	50 / 50	~84 h
<code>make pnr_sdf</code>	Innovus post-route	SDF annotated	50 / 50	~163 h

Post-route simulation runs about $1.1\times$ slower than synthesised because of the added CTS buffers, filler cells, and hold-fix buffers in the post-route netlist. SDF backannotation adds another $1.8\times$ on top: every net carries an RC delay and every flop fires four timing checks (setup, hold, recovery, removal). High-resolution early layers (0–2) dominate all three runs and account for roughly half the wall-clock time.

L4 uses the Innovus-exported SDF (`mfn_frontend_top.v3.final.sdf`, 117MB) backannotated into the same gate-level simulation. After 163h of wall-clock time the full 50-layer run completed with all 50 layers bit-exact against the C-model and zero TCHK.VIOL, setup-violation, or hold-violation events. In the absence of PrimeTime SI this is the strongest signoff we can produce: the netlist runs MobileFaceNet correctly at 200 MHz with real cell and wire delays and live setup/hold checks.

The verification flow itself had one early failure worth recording. The first gate-level run reported 50/50 mismatches even though RTL was 50/50 MATCH. A SystemVerilog `bind`-based probe traced the cause to a testbench race: `pixel_in` was driven on `@(posedge clk)` from the DUT's `x_out` output, but the gate-level netlist has delta-cycle delay on port outputs, so the testbench was latching



the previous-cycle address. Moving the drive to `@(negedge clk)` (after `x_out` has settled) restored all 50 layers. The same testbench is now used for every gate-level and SDF run.

Limitations to keep in mind: this is a single typical-corner signoff (FreePDK45 does not ship multi-corner Liberty), there is no signal-integrity or cross-talk analysis, and the equivalence is shown by simulation rather than formal proof.

8.3 FPGA Verification (Zhang): Simulation and On-Board

Placeholder — Zhang’s FPGA verification results go here. Should at minimum cover: simulator results (vs same C-model golden), on-board measurement methodology and result, and any deviations from the C-model.

9 Ethical Considerations

Face recognition is a sensitive application, and an accelerator that lowers its power and cost is not neutral. Three concerns shape what we built and what we left out: who can be identified, whether the system performs equally across people, and what happens to the biometric data.

Table 18 lists the abuse vectors we considered when choosing the interface. We expose only the 128-d embedding and a match/no-match decision; the raw image is never readable through the chip’s pins. That decision rules out the cheapest forms of mass surveillance (you cannot screenshot people from this hardware) but does not prevent a system integrator from logging the embeddings themselves. The hardware can only constrain the worst options; deployment policy decides the rest.

Table 18: Abuse Vectors and Mitigations

Vector	Risk	Mitigation
Mass surveillance	Cheap edge inference makes installation in public spaces economically trivial.	Hardware cannot self-mitigate. EU AI Act prohibits real-time remote biometric identification in most public contexts.
Coercive unlock	A phone unlocked by face can be opened by pointing it at the owner.	Out of scope; addressed by the OS via liveness detection.
Template leak	Embedding vectors are permanent; unlike passwords they cannot be revoked once exposed.	Embedding-only output; templates stored in a hardware-isolated enclave in a production SoC.
Presentation attack	A photo, mask, or deepfake can fool an embedding-only pipeline.	Requires a separate liveness stage; this accelerator does not address spoofing.
Model substitution	Replacing the bitstream or netlist could bias the accelerator toward specific identities.	Weights loaded from a signed image in OTP or secure flash.

Demographic bias is a known property of face-recognition networks. NIST FRVT 2019 and Buolamwini and Gebru’s 2018 audit both found higher error rates on darker-skinned and female subjects, and quantising to INT16 Q10 does not remove that bias because it is in the trained weights. A deployment built on this accelerator should measure per-demographic error rates on its actual user population, tune the verification threshold to that population, and always offer a non-biometric fallback authentication path.

Privacy compliance is a system-level requirement, not a chip-level one. GDPR Article 9 treats biometric data as special-category personal data, and CCPA has similar provisions. Exposing only the embedding helps but does not satisfy these regulations on its own. The integrator still needs consent capture, purpose limitation, template storage controls, and a documented opt-out path. We treat the chip as necessary but not sufficient.



Putting these together, the deployment patterns we consider acceptable are verification (am I what I claim to be?) rather than identification (who is this stranger?), with user-controlled enrollment, on-device template storage, and an audit trail the user can inspect. Anything broader requires regulatory review that is outside the scope of a hardware project.

10 Conclusions

This project delivered a MobileFaceNet convolution frontend accelerator along two complementary paths: an ASIC in FreePDK45 and an FPGA on Zynq, both verified against the same C-model golden reference. Table 19 consolidates the headline metrics against the original constraints from Section 4.

Table 19: Final Metrics vs Design Constraints

Metric	Constraint	ASIC	FPGA	Met?
Clock frequency	≥ 100 MHz (200 stretch)	200 MHz		✓ / ✓
Throughput	≥ 10 FPS (target)	14.5 FPS		✓
Power	< 100 mW	96.3 mW		✓
Core area	< 1 mm ²	0.518 mm ²	–	✓
DRC / antenna	0 violations	0 / 0	–	✓
50-layer MATCH	required	50/50 (RTL+syn+pnr)		✓
Timing signoff	WNS > 0	+0.089 ns		✓

Did the design meet spec? Yes. All hard constraints were met; both soft constraints (power, area) cleared with margin. The ASIC path delivered $2\times$ the stretch frequency target and stayed under the 100 mW envelope. The functional correctness goal (50/50 layer MATCH against the C-model) was satisfied at every implementation stage, including post-route with SDF backannotation.

What was not delivered.

- No tape-out: the ASIC is design-kit-only.
- No formal LEC: we used full GL simulation instead.
- Single typical-PVT corner; no SI / cross-talk analysis.

Table 20: Suggested Follow-on Work

Priority	Task
High	Multi-corner timing closure (slow / fast / typ); add signal-integrity-aware STA when tools allow.
High	Formal LEC pass (Conformal / Formality) against RTL: seconds of run time replace days of GL simulation.
Medium	Integrate a small DMA / AXI shim and tape out a test chip on an MPW shuttle (FreePDK45 educational; real foundry would substitute its own PDK).
Medium	Replace combinational weight ROM with an external high-bandwidth weight SRAM + AXI-Lite write port for runtime model updates.
Low	Mixed-precision exploration: INT8 activations + INT16 weights could roughly halve switching power while preserving the cosine-similarity gap.

Future work.

Lessons learned. Verification took longer than RTL design. Two specific items dominated the schedule: (i) the gate-level testbench race that required `bind`-based probes to localise (Section 6.1),



and (ii) the OpenRAM LEF patching needed to clean up DRC. Both would have been easier with deterministic instrumentation in place from day one.

References

- [1] Sheng Chen, Yang Liu, Xiang Gao, and Zhen Han. Mobilefacenets: Efficient cnns for accurate real-time face verification on mobile devices, 2018.
- [2] Yu-Hsin Chen, Tushar Krishna, Joel S. Emer, and Vivienne Sze. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE Journal of Solid-State Circuits*, 52(1):127–138, 2017.
- [3] Anjith George, Christophe Ecabert, Hatf Otroschi Shahreza, Ketan Kotwal, and Sebastien Marcel. Edgeface: Efficient face recognition model for edge devices, 2024.
- [4] Linjun SUN, Weijun LI, Xin NING, Liping ZHANG, Xiaoli DONG, and Wei HE. Gradient-enhanced softmax for face recognition. *IEICE Transactions on Information and Systems*, E103.D(5):1185–1189, 2020.